Section 2: Data Structures

Identifying structures from Java code, Big-O complexity, trees, linked lists, stacks/queues, and more

Q23 | 2020 (COVID - single packet)

Once a method has correctly implemented a binary search algorithm, which of these would represent the tightest correct runtime
on an array of n elements?

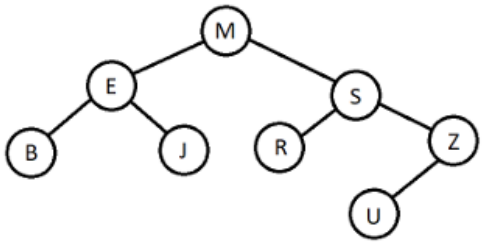
A) O(n2
B) O(n
C) O(1
D) O(log n
E) O(n log n

Q37 | 2020 (COVID - single packet)

Question 37.

Which of the following represents a pre-order traversal of the binary search tree shown on the right?

A) M E B J S R Z U
B) B J E R U Z S M
C) B E J M R S U Z
D) M E S B J R Z U
E) U Z R J B E S M



Question 38.

Question 23.

Which of the following shows the output of this client code?

```
InvB list = new InvB();
list.cat("mercury");list.cat("mars");
list.cat("earth");list.cat("jupiter");
list.bird();list.dog();list.bird();
while(!list.snake())
     out.print(list.dog() + " ");
```

A) `earth mars mercury`

B) `mars earth jupiter`

C) `mercury`

D) `jupiter earth mars`

E) `jupiter`

Question 24.

The class InvB implements a _____.

A) queue

B) map

C) stack

D) set

E) priority queue

```
//Use the class shown here to answer
// questions 23 and 24.
import java.util.*;
public class InvB {

  private ArrayList<String> list;

  public InvB() {
    list = new ArrayList<String>();
  }

  public void cat(String s) {
    list.add(s);
  }

  public String dog() {
    return list.remove(list.size() - 1);
  }

  public String bird() {
    return list.get(list.size() - 1);
  }

  public boolean snake() {
    return list.size() == 0;
  }
}
```

Question 25.

What is the output of the code segment to the right?

A) `five 4`

B) `zero 4`

C) `one 5`

D) `three 5`

E) `two 6`

```
Queue<String> list = new
LinkedList<String>();
list.add("two");list.add("one");
list.add("five");list.add("zero");
list.add("three");list.add("four");
list.poll();list.remove();
out.print(list.peek() + " " + list.size());
```

Q29 | 2020 (COVID - single packet)

Question 27.

Which of the following must replace **<code>** to ensure that the method will compile and execute as intended?

A) `k<=0 && list[k]<current`

B) `k>=0 || list[k]>current`

C) `k>=0 && list[k]>current`

D) `list[k]>current`

E) `k<=0`

Question 28.

Assume that **<code>** has been replaced with the correct code and this client code is executed.

```
int[] list = {6,7,1,8,2,9,0,5,4,3};
insertion(list);
```

What is printed when `i` equals 4 if this line of code

```
out.println(Arrays.toString(list));
```

replaces the comment?

A) `[1, 2, 0, 5, 4, 3, 6, 7, 8, 9]`

B) `[0, 1, 2, 3, 4, 9, 6, 5, 7, 8]`

C) `[1, 6, 7, 8, 2, 9, 0, 5, 4, 3]`

D) `[1, 2, 6, 7, 8, 9, 0, 5, 4, 3]`

E) `[0, 1, 2, 3, 7, 9, 6, 5, 4, 8]`

Question 29.

Once the method `insertion` has been correctly implemented, what is the worst case time complexity for this method?

A) O(1)

B) O(n)

C) O(log n)

D) O(n log n)

E) O(n^2)

```
//Use the following implementation of
//an insertion sort to answer
//questions 27, 28 and 29.

public static void insertion(int[] list)
{
 for(int i = 1; i < list.length;i++)
 {
  int current = list[i];
  int k;
  for(k = i - 1; <code>; k--)
    list[k + 1] = list[k];
  list[k + 1] = current;
  //comment
 }
}
```

Q40 | 2020 (COVID - single packet)

If the values shown here are placed into a binary search tree in the order that they are listed, which value will serve as the root
node? Write your answer in the blank provided on the answer document.
2 0 8 4 6 7 1 3 5 9

Question 31

Assume that the elements to the right are inserted into an Unbalanced Binary Search Tree where duplicate elements are **not added** to the tree.

How many internal nodes will the tree have?

A) 15 **B)** 13

C) 10 **D)** 16

E) 9

Question 32

Under the same assumption as Question 31, how many leaf nodes will the tree have?

A) 1 **B)** 5

C) 3 **D)** 4

E) 7

Question 33

Under the same assumption as Question 31, what is the diameter of the tree?

A) 8 **B)** 6

C) 9 **D)** 10

E) 4

Question 34

Under the same assumption as Question 31, what is the worst-case time complexity for the operation `search()` in an Unbalanced Binary Search Tree? You may assume that n is the number of elements in the tree.

A) $O((n))$ **B)** $O(n^2)$

C) $O(n)$ **D)** $O(\sqrt{n})$

E) $O((n))$

```
34, 86, 28, 29, 33, 14, 52, 31,
92, 14, 15, 92, 31, 92, 105, 95,
97, 118
```

Q35 | 2025 Invitational A

Question 35

Which of the following could replace **<1*>** to ensure that any classes that are stored as data within this data structure are compatible with the `Comparable` interface?

A) `extends` **B)** `implements`

C) `requires` **D)** Either A or B.

E) None of the above.

Question 36

Which of the following lines of code could replace **<2*>** so that the function `peek()` properly returns the value of the data stored in `head`?

A) `return head.data`

B) `return this.head.data`

C) `return this.head`

D) Either A or B.

E) All of the above.

Question 37

Which of the following well-known data structures is the class `DataStruct` an implementation of?

A) `LinkedList` **B)** `Queue`

C) `Stack` **D)** `Vector`

E) `Deque`

Question 38

Which of the following classes would not be able to be stored within this data structure?

A) `Integer` **B)** `String`

C) `BigInteger` **D)** `double[]`

E) None of the above.

```
public class DataStruct<T <1*> Comparable<T>> {
    private class Node {
          public T data;

          public Node next;

          public Node(T d, Node n) {
                this.data = d;
                this.next = n;
          }
    }

    private Node head;

    private int size;

    public DataStruct() {
          this.size = 0;
          this.head = null;
    }

    public T peek() {
          if(head == null) {
                return null;
          }
          <2*>
    }

    public T pop() {
          T data = this.head.data;
          this.head = this.head.next;
          this.size--;
          return data;
    }

    public T push(T data) {
          Node newHead = new
                Node(data, this.head);
          this.head = newHead;
          this.size++;
          return data;
    }

    public int size() {
          return size;
    }
}
```

Q40 | 2025 Invitational B

What is the in-order traversal of the binary search tree created by inserting the following values in order?
34 67 212 17 9 6 104 8 29 48 97 147 1

Question 31

What can replace **<1*>** in the code to the right so that any object which implements the `Comparable` interface can be stored within the new object `DataStructure`?

A) `T`

B) `Comparable`

C) `? extends Comparable`

D) `T extends Comparable<T>`

E) None of the above

Question 32

Assuming that **<1*>** has been filled in properly, what can replace **<2*>** in the code to the right so that any object which implements the `Comparable` interface can be stored within the new object `DataStructure`?

A) `T`

B) `Comparable`

C) `? extends Comparable`

D) `T extends Comparable<T>`

E) None of the above

Question 33

What is the tightest upper bound on the time complexity of the `extract()` method in the code to the right?

A) $\mathcal{O}(n)$

B) $\mathcal{O}(n \log_2 n)$

C) $\mathcal{O}(\sqrt{n})$

D) $\mathcal{O}(1)$

E) $\mathcal{O}(\log_2 n)$

Question 34

```
class DataStructure<<1*>> {
    private ArrayList<<2*>> arr;

    public DataStructure(int n) {
        arr = new ArrayList<<2*>>();
    }

    private void swap(int i, int j) {
        <2*> temp = arr.get(i);
        arr.set(i, arr.get(j));
        arr.set(j, temp);
    }

    public void insert(<2*> val) {
        int i = arr.size();
        int parent = (i - 1) / 2;
        arr.add(val);

        while (i > 0 && arr.get(i).compareTo(
            arr.get(parent)) < 0)
        {
            swap(i, parent);
            i = parent;
            parent = (i - 1) / 2;
        }
    }

    public <2*> extract() {
        if (arr.isEmpty()) {
            return null;
        }

        <2*> min = arr.get(0);
        <2*> last = arr.remove(arr.size() - 1);
        if (!arr.isEmpty()) {
            arr.set(0, last);

            int i = 0;
            while (true) {
                int l = (2 * i) + 1;
                int r = (2 * i) + 2;
                int minIndex = i;

                if (l < arr.size() && arr.get(l)
                    .compareTo(arr.get(minIndex))
                    < 0)
```

Question 34

The class `DataStructure` in the code to the right is an implementation of what well-known data structure?

A) Dequeue

B) AVL Tree

C) Binary Search Tree

D) Min-Heap

E) Max-Heap

```
                < 0)
            {
                minIndex = l;
            }
            if (r < arr.size() && arr.get(r)
                .compareTo(arr.get(minIndex))
                < 0)
            {
                minIndex = r;
            }

            if (minIndex == i) {
                break;
            }
            swap(i, minIndex);
            i = minIndex;
        }
    }
    return min;
  }
}
```

Q35-38 | 2025 Region

Q18-19 | 2025 State

Question 18

What is the asymptotic time complexity of the code to the right?

A) $\mathcal{O}(n \log n)$ **B)** $\mathcal{O}(\log n)$

C) $\mathcal{O}(n)$ **D)** $\mathcal{O}(n^2)$

E) $\mathcal{O}(n^2 \log n)$

Question 19

Which data structure is demonstrated by the code to the right?

A) Queue **B)** Min-Heap

C) Linked List **D)** Max-Heap

E) Stack

```
PriorityQueue<Integer> pq;
pq = new PriorityQueue<Integer>
    (Collections.reverseOrder());
for(int y = 0; y < N; y++) {
    double d = Math.random() * 100;
    pq.add((int)(d));
}
out.println(pq);
```

Q24 | 2025 State

```
class DataStructure<T> {
    private class Node<1*> { // EXCEPT THIS LINE
        public T data;
        public Node next, prev;

        public Node(T data) { // EXCEPT THIS LINE
            this.data = data;
        }
    }

    private Node head, tail;
    private int size;

    public DataStructure() {
        head = tail = null;
        size = 0;
    }

    public void addFront(T data) {
        Node newNode = new Node(data);
        if (head == null) {
            tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
        }
        head = newNode;
        size++;
    }
```

```
public void addBack(T data) {
    Node newNode = new Node(data);
    if (tail == null) {
        head = newNode;
    } else {
        newNode.prev = tail;
        tail.next = newNode;
    }
    tail = newNode;
    size++;
}

public T removeFront() {
    if (head == null) {
        return null;
    }
    T data = head.data;
    head = head.next;
    if (head == null) {
        tail = null;
    } else {
        head.prev = null;
    }
    size--;
    return data;
}

public T removeBack() {
    if (tail == null) {
        return null;
    }
    T data = tail.data;
    tail = tail.prev;
    if (tail == null) {
        head = null;
    } else {
        tail.next = null;
    }
    size--;
    return data;
}
```

24. The class DataStructure in the code to the right is a partial implementation of what well-known data structure?

A) Dequeue
B) AVL Tree
C) Stack

D) Min-Heap
E) Queue

Q36 | 2024 Invitational A

Question 36

Assume that a binary search tree using the letters B A C has a height of 1. Insert the letters to the right into an empty binary search tree. What is the height?

Note: Duplicate values **ARE** added to the binary tree. Duplicate values are added to the left of the original occurrence. This tree will have 16 nodes.

A) 4 **B)** 5 **C)** 6 **D)** 7 **E)** 8

U I L I N V I T A T I O N A L A

Q32 | 2024 Invitational B

How many levels?
A) 8
B) 10
C) 11
D) 12
E) 1999

In a Binary Search Tree consisting of 2000 nodes. What is the minimum number of levels the tree may have? Example: An initially empty Binary Search Tree adding the elements B, then A, then C would have 2 levels.

Q39 | 2024 District

Imagine you have an empty binary search tree. First, you add the five vowels into the tree, one at a time in this order (A,E,I,O,U). Next, you add to the tree the remaining consonants in alphabetical order, starting with B and ending with Z. How many leafs would the tree have?
Write the number in blank #39.

Q40 | 2024 State

Create a binary search tree inserting the letters "AUSTIN". Start with A and end with N. How many leaves will the tree have? Write your answer in blank #40.

Q36 | 2023 Invitational A

If the letters to the right were inserted into an initially empty binary search tree in the order shown, how many leaves would the resulting tree contain?

A) 9
B) 10
C) 12
D) 1
E) 19

A B C D E F G H I J J I H G F E D C B A

Q40 | 2023 District

Create the binary search tree using the letters in the word to the right. After building the tree, determine how many leaf nodes the tree contains.

T R A M P O L I N E

Q40 | 2023 Region

Consider a binary tree with 1023 nodes. What is the greatest number of leaves that the tree could contain?

Q21 | 2022 Invitational A

```
public static void M2(int n){
   for(int i = 0;i<n;i++)
   {
      for(int j = 1;j<n;j*=2)
      {
         out.print("*");
      }
   }
}
```

What is the tightest upper bound for the Big O complexity of M2?

A) O (n
B) O (nlogn
C) O (n2
D) O (2n
E) none of the above

```
private class DataStructure<E>{
   private class O{
      E elem;
      O next;
      public O(E e, O n)
      {
         this.elem = e;
         this.next = n;
      }
   }

   O h;
   int sz;

   public void add(E e)
   {
      h = new O(e, h);
      sz++;
   }

   public E next()
   {
      return h.elem;
   }

   public E remove()
   {
      E e = next();
      h = h.next;
      sz--;
      return e;
   }

   public boolean isEmpty() {
      return sz == 0;
   }
 }
```

Q29 | 2022 Invitational A

What data structure does DataStructure represent?

A) Queue
B) PriorityQueue
C) Stack
D) Set
E) none of the above

Q18 | 2022 Invitational B

D) 55

E) 5555

Question 18

What is the most restrictive Big O upper bound of `f1(n)`?

A) O(n)

B) O(nlogn)

C) $O(n^2)$

D) $O(2^n)$

E) O(1)

Question 19

What is the most restrictive Big O upper bound of `f2(n)`?

A) O(n)

B) O(nlogn)

C) $O(n^2)$

D) $O(2^n)$

E) O(1)

```
public static int f1(int x){
   if(x < 2)
      return x;
   return f1(x-1) + f1(x-2);
}

public static int f2(int x)
{
   int[] f = new int[x+1];
   f[0] = 0;
   f[1] = 1;
   for(int i = 2;i<f.length;i++)
      f[i] = f[i-1] + f[i-2];
   return f[x];
}
```

Q29 | 2022 Invitational B

Question 27

What is output by the following client code?

```
DataStructure<Integer> ds;
ds = new DataStructure<>();
for(int i: new int[] {1,5,2,7,4,8,9})
   ds.add(i);
while(!ds.isEmpty())
   out.print(ds.remove()+" ");
```

A) 1 5 2 7 4 8 9

B) 9 8 4 7 2 5 1

C) 1 2 4 5 7 8 9

D) 9 8 7 5 4 2 1

E) There is no output due to a runtime error

Question 28

What is output by the following client code?

```
DataStructure<Integer> ds;
ds = new DataStructure<>();
for(int i: new int[] {1,5,2,7,4,8,9})
   ds.add(i);
while(!ds.isEmpty())
   ds.remove();
out.print(ds.next());
```

A) 0

B) o

C) null

D) 1

E) There is no output due to a runtime error

Question 29

What data structure does DataStructure represent?

A) Queue

B) PriorityQueue

C) Stack

D) Set

E) none of the above

```
public class DataStructure<E>{
   private class O{
      E e;
      O o;
      public O(E e, O o)
      {
         this.e = e;
         this.o = o;
      }
   }

   O o;
   O t;
   int sz;

   public void add(E e)
   {
      if(o == null) {
         o = t = new O(e,null);
      }
      else{
         t = t.o = new O(e, null);
      }
      sz++;
   }

   public E next()
   {
      return o.e;
   }

   public E remove()
   {
      E e = next();
      o = o.o;
      sz--;
      return e;
   }

   public boolean isEmpty() {
      return sz == 0;
   }
}
```

Q23 | 2022 District

Question 24.

```
public static int rec(String a, String b,
                              int ai, int bi){
  if(ai == a.length() ||
     bi == b.length()){
     return 0;
  }
  if(a.charAt(ai) == b.charAt(bi))
  {
     return 1 + rec(a,b,ai+1,bi+1);
  }
  int x = rec(a,b,ai+1,bi);
  int y = rec(a,b,ai,bi+1);
  return Math.max(x,y);
}
```

Question 24.

Assume that a and b are strings of length N. Which of the following is the most restrictive Big O upper bound of method call `rec(a, b, 0, 0)`?

A) O(N)

B) O(N^2)

C) O(N^3)

D) O(2^N)

E) None of the above

Q25 | 2022 District

```
public static int dp(String a, String b)
{
   int al = a.length();
   int bl = b.length();
   int[][] dp = new int[al+ 1][bl + 1];
   for(int i = 0;i<al;i++)
   {
      for(int j = 0;j<bl;j++)
      {
         if(a.charAt(i) == b.charAt(j))
         {
            dp[i+1][j+1] = dp[i][j] + 1;
         }else{
            int x = dp[i][j+1];
            int y = dp[i+1][j];
            dp[i+1][j+1] = Math.max(x,y);
         }
      }
   }
   return dp[al][bl];
}
```

Question 25.

Assume that a and b are strings of length N. Which of the following is the most restrictive Big O upper bound of method call `dp(a, b)`?

A) $O(N)$

B) $O(N^2)$

C) $O(N^3)$

D) $O(2^N)$

E) None of the above

Q34 | 2022 District

Question 34.

The letters in "TEXASUIL" are inserted into a binary search tree in order. What is the post-order traversal of the resultant tree?

A) AEILSTUX

B) TEXASUIL

C) ALISEUXT

D) AEILSTUX

E) None of the above

Q37 | 2022 District

```
public class Structure<T>{
   int cap;
   int sz;
   HashMap<Integer, LinkedList<T>> map;

   public Structure(int cap)
   {
     this.cap = cap;
     map = new HashMap<>();
   }

   public void add(T t)
   {
     if(contains(t))
       return;
     int i = t.hashCode() % cap;
     if(!map.containsKey(i))
       map.put(i, new LinkedList<T>());
     map.get(i).add(t);
     sz++;
```

```
}

public boolean contains(T t)
{
  int i = t.hashCode() % cap;
  if(!map.containsKey(i))
  return false;
  for(T x: map.get(i))
    if(x.equals(t))
      return true;
  return false;
}

public int size()
{
  return sz;
}
```

37.

Which of the following is the most restrictive Big O upper bound
of method add?

A) O(1
B) O(log(N
C) O(N
D) O(N2
E) None of the above
}

Q38 | 2022 District

What data structure does the Structure class represent?

A) Map
B) HashMap
C) Queue
D) ArrayList
E) HashSet

Q40 | 2022 District

If a binary tree contains 12 nodes, what is the most number of levels (including the root) that tree might have? Write your answer
in the blank provided on the answer document.

2022 Region

Question 21.

What is the output of the client code below?

```
int[] ints = new int[]{1,4,1,5,1,1,7};
out.print(dp(ints));
```

A) 1

B) 3

C) 4

D) 5

E) 7

Question 22.

What is the output of the client code below?

```
int[] ints = new int[]{3,1,4,1,5,9};
out.print(rec(ints, 0));
```

A) 1

B) 3

C) 4

D) 5

E) 6

Question 23.

Which of the following is the most restrictive Big O upper bound of method call `dp(ints)`? Assume `ints` is of length `N`

A) O(N)

B) O(NlogN)

C) O(N^2)

D) O(N^3)

E) O(2^N)

Question 24.

Which of the following is the most restrictive Big O upper bound of method call `rec(ints)`? Assume `ints` is of length `N`

A) O(N)

B) O(NlogN)

C) O(N^2)

D) O(N^3)

E) None of the above

```
public static int dp(int[] ints)
{
  int[] dp = new int[ints.length];
  Arrays.fill(dp, 1);
  int ret = 0;
  for(int i = 1;i<dp.length;i++)
  {
    for(int j = i-1;j>=0;j--)
    {
      if(ints[j] <= ints[i])
      {
        dp[i] = Math.max(dp[i],dp[j] + 1);
      }
    }
    ret = Math.max(ret, dp[i]);
  }
  return ret;
}

public static int rec(int[] ints, int ind)
{
  int r = 1;
  for(int i = ind+1;i<ints.length;i++)
  {
    if(ints[i] >= ints[ind])
    r = Math.max(r, 1 + rec(ints, i));
  }
  return r;
}
```

Q29 | 2022 Region

What is the worst case time complexity of add() for Java's PriorityQueue? Assume comparison is an O(1) operation.

A) O(1
B) O(log(N
C) O(N
D) O(NlogN
E) O(N2

Q31 | 2022 Region

What is the best case time complexity of bubble sort?

A) O(1
B) O(N
C) O(NlogN
D) O(N^2
E) None of the above

Q35 | 2022 Region

The letters “DBFCAGE” are inserted into a binary search tree in order. Which of the following traversals will result in a visitation
order of DBFACEG?

A) Pre-order traversal
B) Post-order traversal
C) In-order traversal
D) Level-order traversal
E) None of the above

Q17 | 2022 State

What is the worst case Big O Complexity of java.util.PriorityQueue remove() method? Assume comparison is an O(1)
operation.

A) O(1
B) O(log(N
C) O(N
D) O(Nlog(N
E) O(N2

Q40 | 2021 Invitational A

Write the signed 8-bit binary two's complement representation of -104 in the blank provided on the answer document.

| 2021 Invitational B

Question 23.

Which of the following shows the output of the class `InvB` shown on the right?

A) `dog cat bird turtle pig rat dog dog`

B) `cat bird turtle pig rat dog cat`

C) `turtle pig rat dog bird`

D) `cat dog rat pig turtle bird cat`

E) `bird dog pig rat dog turtle`

Question 24.

The class `InvB` implements a _____.

A) queue

B) map

C) stack

D) set

E) priority queue

```
//Use the class shown here to answer
// questions 23 and 24.
import java.util.*;
public class InvB {
  private ArrayList<String> list;

  public InvB() {
    list = new ArrayList<String>();
  }

  public void abc(String s) {
    list.add(s);
  }

  public String xyz() {
    return list.remove(0);
  }

  public String mno() {
    return list.get(0);
  }

  public boolean efg() {
    return list.size() == 0;
  }

  public static void main(String[] args) {
  InvB obj = new InvB();
  obj.abc("dog");obj.abc("cat");
  obj.abc("bird");obj.abc("turtle");
  obj.abc("pig");obj.abc("rat");
  obj.mno();obj.xyz();obj.abc("dog");
  obj.abc(obj.mno());
  while(!obj.efg())
    System.out.print(obj.xyz() + " ");
  }
}
```

Question 25.

```
Map<String,String> m = new
```

Q: 27-29 | 2021 Invitational B

Question 27.

Which of the following must replace **<code>** to ensure that the method will compile and execute as intended?

A) `j = k`

B) `k = i`

C) `k = 0`

D) `j = i`

E) `k = j`

Question 28.

Assume that **<code>** has been replaced with the correct code and this client code is executed.

```
int[] list = {6,7,1,8,2,9,0,5,4,3};
sort(list);
```

What is printed when `i` equals 4 if this line of code

```
out.println(Arrays.toString(list));
```

replaces the comment?

A) `[1, 2, 0, 5, 4, 3, 6, 7, 8, 9]`

B) `[0, 1, 2, 3, 4, 9, 6, 5, 7, 8]`

C) `[1, 6, 7, 8, 2, 9, 0, 5, 4, 3]`

D) `[1, 2, 6, 7, 8, 9, 0, 5, 4, 3]`

E) `[0, 1, 2, 3, 7, 9, 6, 5, 4, 8]`

Question 29.

Once the method `sort` has been correctly implemented, what is the worst case time complexity for this method?

A) O(1)

B) O(n)

C) O(log n)

D) O(n log n)

E) $O(n^2)$

```
//Use the following implementation of
//a selection sort to answer
//questions 27, 28 and 29.

public static void sort(int[] list)
{
  for(int i = 0; i < list.length - 1; i++)
  {
    int k = i;
    for(int j = i + 1; j < list.length; j++)
      if(list[j] < list[k])
        <code>;
    int t = list[i];
    list[i] = list[k];
    list[k] = t;
    //comment
  }
}
```

Q32 | 2021 Invitational B

What is the average time complexity for insertion into a somewhat balanced binary search tree?

A) O(n

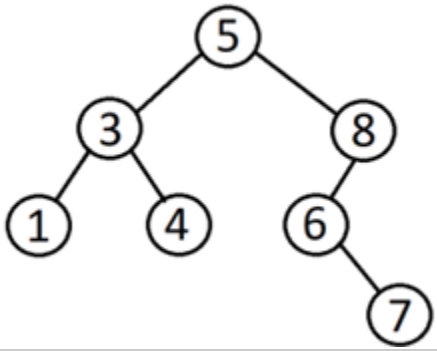
B) O(2n

C) O(n^2)

D) O(log n)

E) O(n log n)

Q36 | 2021 Invitational B



Which of the following represents a post order traversal of the binary search tree shown on the right?

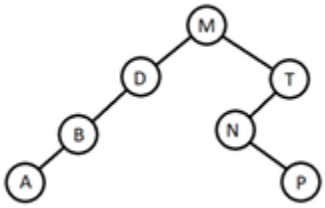
A) 7 1 4 6 3 8 5
B) 1 3 4 5 6 7 8
C) 5 3 1 4 8 6 7
D) 1 4 3 7 6 8 5
E) 5 3 8 1 4 6 7

Q37 | 2021 District

Question 37.

If an F is inserted into the binary search tree shown on the right, which of the following best describes where it will be placed?

A) As the root node
B) As the right node of D
C) As the right node of B
D) As the left node of N
E) As the right node of T



Q40 | 2021 District

If a binary tree contains 255 nodes, what is the least number of levels (including the root) that tree might have?

Q37 | 2021 Region

Question 37.

The `TreeNode` and `BST` classes shown on the right are an incomplete implementation of a binary search tree data structure. Which of the following best describes the function and purpose of `method(E e)`?

A) Deletes `e` from the tree if present and returns `true` or returns `false` if the element is not present.

B) Inserts `e` into the tree and returns `true` or returns `false` if the element is already present.

C) Returns `true` if the tree is a complete binary tree and `false` if not.

D) Returns `true` if a preorder traversal can be completed or `false` if not.

E) Returns `true` if `e` is found in the tree and `false` if not.

```
public class TreeNode<E> {
      protected E element;
      protected TreeNode<E> left;
      protected TreeNode<E> right;

      public TreeNode(E e) {
            element = e;
      }
}

public class BST <E extends Comparable<E>>{

 protected TreeNode<E> root;
 protected int size = 0;
 //Constructors are present but not shown.

 public boolean method(E e) {
   TreeNode<E> current = root;
   while(current != null) {
     if(e.compareTo(current.element) < 0)
       current = current.left;
     else if(e.compareTo(current.element) > 0)
       current = current.right;
     else
       return true;
    }
   return false;
   }
}
```

Q40 | 2021 Region

What is the worst case time complexity for access to an element within an array. Write your answer using Big-O notation in the
blank provided on the answer document.

Q24 | 2019 Invitational A

Question 22.

The method shown on the right is intended to implement a binary search algorithm. Which of the following must replace **<code 1>** both places it occurs to ensure the method will compile and execute as intended?

A) `list.length/2`

B) `(front+back)/2`

C) `list.length-1/2`

D) `(front+list.length)/2`

E) `front+back/2`

Question 23.

Which of the following must replace **<code 2>** to ensure the method will compile and execute as intended?

A) `i`

B) `mid`

C) `front`

D) `back`

E) No additional code is required.

Question 24.

Once implemented correctly, and if n is the length of the array, which of the following is the strictest correct runtime?

A) O(1)

B) O(n)

C) $O(n^2)$

D) O(log n)

E) O(n log n)

```
//Use the following to answer questions 22,
//23 and 24.
public static int bs(int[] list,int e) {
      int i=-1;
      int front=0,back=list.length-1;
      int mid=<code 1>;
      while(back>=front) {
            if(list[mid]==e) {
                  i=mid;
                  break;
            }
            else if(e<list[mid])
                  back=mid-1;
            else
                  front=mid+1;
            mid=<code 1>;
      }
      return <code 2>;
}
```

Q36 | 2019 Invitational A

Question 36.

Which of the following represents a post-order traversal of the binary search tree illustrated on the right?

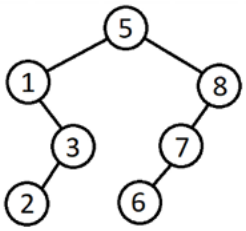
A) 5 1 3 2 8 7 6

B) 2 3 1 6 7 8 5

C) 1 2 3 5 6 7 8

D) 5 1 8 3 7 2 6

E) 8 7 6 2 3 1 5



Q26-29 | 2019 Invitational B

Question 26.

Which of the following must replace **<code>** in the method shown on the right to ensure that the array list is sorted in ascending order?

A) `list[j]>x`

B) `list[i]<x`

C) `list[j]<y`

D) `x<y`

E) `list[j]<x`

Question 27.

Assume **<code>** *has been filled in correctly to answer questions 27, 28 and 29.*

What is printed by the line of code following the comment when `i` is equal to 2 if `list` is initialized as follows?

```
int[] list= {2,9,3,7,0,1,4};
```

A) `[0, 1, 2, 7, 3, 9, 4]`

B) `[9, 7, 4, 2, 0, 1, 3]`

C) `[2, 3, 9, 7, 0, 1, 4]`

D) `[0, 1, 2, 3, 7, 9, 4]`

E) `[2, 3, 7, 9, 0, 1, 4]`

Question 28.

Which of the following algorithms is implemented by the `sort` method shown on the right?

A) Index Sort

B) Selection Sort

C) Radix Sort

D) Insertion Sort

E) Merge Sort

Question 29.

What is the worst-case run time efficiency for the method `sort` where n is the size of array `list`?

A) O(1)

B) O(n log n)

C) O(n)

D) O(log n)

E) $O(n^2)$

```
//Use this method to answer questions
//26 - 29.
public static void sort(int[] list) {
  int x,y;
  for(int i=0;i<list.length;i++) {
    x=list[i];
    y=i;
    for(int j=i+1;j<list.length;j++) {
      if(<code>) {
        x=list[j];
        y=j;
      }
    }
    list[y]=list[i];
    list[i]=x;
    //comment
    out.println(Arrays.toString(list));
  }
}
```

Question 36.

Which of the following Boolean expressions is not equal to A?

A) $A + A$ **B)** $A + 1$ **C)** $A * 1$ **D)** $A * A$ **E)** $A + 0$

Question 37.

Given the class `DataStructure` shown on the right, which of the following data structures must the class be implementing?

A) A set

B) A stack

C) A queue

D) A linked list

E) A tree

```
import java.util.ArrayList;

public class DataStructure {

      private ArrayList<String> list;

      public DataStructure() {
            list=new ArrayList<String>();
      }

      public boolean add(String s) {
            if(!list.contains(s)) {
                  list.add(s);
                  return true;
            }
            else
                  return false;
      }

      public boolean remove(String s) {
            return list.remove(s);
      }
}
```

Q40 | 2019 Invitational B

How many leaves are contained within a binary search tree that is constructed by inserting the following values in the order
shown?
58 19 18 3 4 1 67 60 82

Q25 | 2019 District

What is the run time complexity (Big O value) for an ascending insertion sort of an array that contains n elements if the array is
already sorted in ascending order?

A) O(1)
B) O(n)
C) O(n2)
D) O(log n)
E) O(n log n)

Q32 | 2019 Region

Which of the following run time efficiencies is the fastest for very large values of n?

A) O(n log n
B) O(log n
C) O(n)
D) O(n^2)
E) O(2n)

Q33 | 2019 State

What is the worst case relative run time efficiency (complexity) of a Merge sort ?

A) O(n)
B) O(n^2)
C) O(log n)
D) O(n log n)
E) O(n^3)

Q34 | 2018 Invitational A

```
out.print(sum);
```

Question 32.

Which of the following shows the order of the elements in array `a` when code execution reaches the comment statement and `i` equals 3 given the following client code?

```
int[] a= {3,2,4,1,0};
sort(a);
```

A) [1, 2, 3, 4, 0]
B) [2, 3, 4, 1, 0]
C) [0, 1, 2, 3, 4]
D) [1, 0, 2, 3, 4]
E) [0, 1, 2, 4, 3]

Question 33.

Which of the following sorting algorithms is implemented by the sort method shown on the right?

A) selection sort
B) insertion sort
C) bubble sort
D) quick sort
E) merge sort

```
//Use the following to answer
//questions 32, 33 and 34.
public static void sort(int[] a) {
  for(int i=1;i<a.length;i++) {
    int ce=a[i];
    int k;
    for(k=i-1;k>=0&&a[k]>ce;k--) {
      a[k+1]=a[k];
    }
    a[k+1]=ce;
    //comment
  }
}
```

Question 34.

What is the worst case run time efficiency (Big O value) for the sort method shown on the right?

A) O(n)
B) O(n^2)
C) O(n^3)
D) O(log n)
E) O(n log n)

Question 35.

Q38 | 2018 Invitational A

What is the worst case time complexity for accessing an element in a linked list?

A) O(1)
B) O(log n)
C) O(n) D) O(n^2) E) O(n log n)

Q23-27 | 2018 Invitational B

```
//Use the class shown here to answer questions 23 - 27.

import java.util.ArrayList;

public class DataStructure<E> {

      private ArrayList<E> list=new ArrayList<E>();

      public int size() {
            return list.size();
      }

      public E next() {
            return list.get(size()-1);
      }

      public void add(E item) {
            list.add(item);
      }

      public E remove() {
            return list.remove(size()-1);
      }

      public boolean isEmpty() {
            return list.isEmpty();
      }

      public String toString() {
            return list.toString();
      }
}
```

Question 23.

Which of the following can replace **?????** in the code segment shown on the right and ensure that the remainder of the code will compile and execute correctly?

A) `ArrayList`

B) `Object`

C) `String`

D) A, B, and C

E) B and C

Question 24.

What is printed by **line #1** in the code segment to the right?

A) `snow`

B) `hail`

C) `rain`

D) `dust`

E) `wind`

Question 25.

What is the output of the while loop shown between **comment #1** and **#2** in the code segment to the right?

A) `sleet rain snow wind`

B) `sleet wind snow rain`

C) `sleet hail wind snow rain`

D) `rain snow wind hail sleet`

E) `rain snow wind sleet`

Question 26.

What is printed by **line #2** in the code segment to the right?

A) 0

B) 1

C) 2

D) 3

E) 4

```
DataStructure<?????> ds=new
DataStructure<?????>();
ds.add("rain");
ds.add("snow");
ds.add("wind");
ds.add("hail");
ds.add("dust");
ds.remove();
out.println(ds.next());//line #1
ds.add("sleet");
//comment #1
while(!ds.isEmpty())
      out.print(ds.remove()+" ");
//comment #2
out.println("\n"+ds.size());//line #2
```

Question 27.

Which of the following data structures does the class DataStructure, shown above, implement?

A) set **B)** linked list **C)** priority queue **D)** stack **E)** queue

Which of the following data structures does the class DataStructure, shown above, implement?
A) set
B) linked list
C) priority queue
D) stack
E) queue

Q37 | 2018 Invitational B

Each element within a linked list data structure is stored within a _____?

A) array
B) edge
C) identifier
D) class
E) node

Q27-29 | 2018 District

Question 27.

Which of the following must replace **<code>** in the method shown on the right to ensure the method will sort `a` in ascending order?

A) `k>=0&&a[k]<ce`

B) `k>=0&&a[k]>ce`

C) `k>=0||a[k]>ce`

D) `k>=i&&a[k]>ce`

E) `k>=ce&&a[i]>ce`

Question 28.

Once **<code>** has been filled in correctly, which of the following sorting algorithms is implemented by the `uilSort` method?

A) heap sort

B) quick sort

C) insertion sort

D) selection sort

E) merge sort

Question 29.

What is the least restrictive worst case time efficiency (Big O value) for the `uilSort` method?

A) O(1)

B) O(n)

C) $O(n^2)$

D) O(log n)

E) O(n log n)

Question 30.

```
//Use the following method to answer
//questions 27, 28, 29 and 30.
public static void uilSort(int[] a) {
  int i=1;
  do {
    int ce=a[i];
    int k=i-1;
    while(<code>) {
      a[k+1]=a[k];
      k--;
      }
    a[k+1]=ce;
    //comment
    i++;
  }while(i<a.length);
}
```

Data Structures

2020 (COVID - single packet)

Q23: D Q37: A

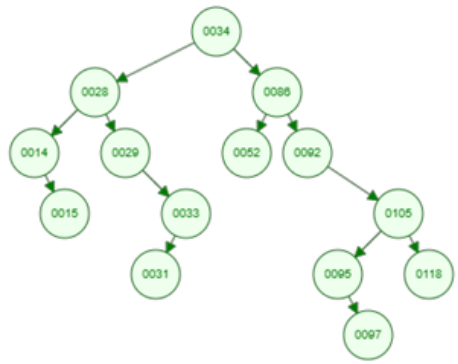
23.	A	The class InvB implements a stack. The cat method is push. The dog method is pop. The bird method is peek and snake is the isEmpty method. Elements are added and removed in a first in, last out fashion.
24.	C	See #23.
25.	A	poll() retrieves and removes the element at the head of the queue. A queue adds and removes elements in a first in, first out fashion.

27.	C	The inner for loop is searching for the proper place to insert the current element by working back through the elements that are already in place searching for the first element that is less than the current element.
28.	D	i = 1 [6, 7, 1, 8, 2, 9, 0, 5, 4, 3] i = 2 [1, 6, 7, 8, 2, 9, 0, 5, 4, 3] i = 3 [1, 6, 7, 8, 2, 9, 0, 5, 4, 3] i = 4 [1, 2, 6, 7, 8, 9, 0, 5, 4, 3]
29.	E	best O(n) average O(n^2) worst O(n^2)

Q40: 2

2025 Invitational A

Q31: E

31.	E	A copy of the binary search tree has been provided below:  Internal nodes are those nodes that have 1 or more children. There are a total of 9 nodes that have 1 or more child (nodes 34, 28, 14, 29, 33, 86, 92, 105, and 95).

32.	B	Leaf nodes are those nodes that have no children of their own. There are a total of 5 nodes that have no children of their own (nodes 15, 31, 52, 97, and 118).
33.	C	The diameter of a tree is the greatest number of edges between any two nodes within the tree. In the case of this tree, those nodes are 31 and 97, which have 9 edges between them.
34.	C	The worst-case scenario for an Unbalanced Binary Search Tree is that the nodes are inserted in sorted order. This effectively creates a Linked List, which has a linear time complexity for search operations.

35.	A	The key word `extends` is the only one among the ones listed which allows to specify which interfaces or classes the generic types must implement or inherit. This is commonly confused with the `implements` key word, which is used when declaring a class or interface that should express the behaviors of another interface. The `requires` key word, while a valid key word in the Java library, denotes a required library within a module, and thus is irrelevant to this question.
36.	D	Since there are no local versions of the variable `head`, either option A or option B will reference the global `head` variable and perform the function as intended. Option C will break since the return type of the function is of type `T` and not of provided type `Node`.
37.	C	This is a `Stack` since it denotes the methods `peek()`, `pop()`, and `push()` and has the FIFO (First-in-First-out) property with regards to how elements are handled. While the FIFO property *can* also be obtained using a `Deque`, for this to be a `Deque`, we would need to have separate methods for peeking, pushing, and popping elements from both sides of the `Queue`, which is not present in the provided implementation.
38.	D	`Integer`, `String`, and `BigInteger` all implement the `Comparable` interface. While `double[]` *can* actually be stored in a normal `Stack`, since `double[]` does not implement the `Comparable` interface, it cannot be stored in *this* implementation of a `Stack`.

2025 Invitational B

		Add 1: 10011001
40.	`1 6 8 9 17 29 34 48 67 97 104 147 212`	
	In order for binary search trees is just the sorted order, so you don't even need to make the tree.	

2025 Region

		there is only one error as mentioned).
31.	D	This will require the use of generics, hence the `T`. Moreover, since the problem asks that objects implement the `Comparable` interface, we need to append `extends Comparable<T>`.
32.	A	Note that we don't have to specify `T extends Comparable<T>` for every other reference after having declared the properties of `T`. Afterwards, we can just refer to the generic type `T`, since the compiler knows that `T` implements the `Comparable` interface by what we did in question 33.
33.	E	The `extract` method is really the `extractMin` method from a heap algorithm, which means that it runs in $\mathcal{O}(\log_2 n)$ time. Note that the loop will run at most $\mathcal{O}(\log_2 n)$ time since, after each iteration, we cut down the search space by a factor of 2.
34.	D	As previously stated, this is a heap, but since the element removed by the heap when the extract method is called is the smallest value, that makes it a min-heap. Note that all data structures mentioned (except for option A) do have a tree-like structure, the only one that satisfies all properties of the data structure is a min-heap.

35.	D	With an array, we can do $\mathcal{O}(1)$ random access of the list, which means that we can perform binary search on the list. Binary search takes $\mathcal{O}(\log_2 n)$ time, where n is the array size. The input size is $m = n^2$, so the total complexity is $\mathcal{O}(\log_2 n^2) = \mathcal{O}(2\log_2 n) = \mathcal{O}(\log_2 n)$
36.	A	With a singly-linked list, we cannot do $\mathcal{O}(1)$ random access, and have to do a linear search for elements. Since the input size is $m = n^2$, the total time complexity is $\mathcal{O}(n^2)$.
37.	A	Singleton is a creational design pattern that lets you ensure that a class has only one instance, while providing a global access point to this instance.
38.	C	Despite having an average time complexity of $\mathcal{O}(n\log_2 n)$, Quick Sort is the only algorithm among the options that has a worst-case time complexity of $\mathcal{O}(n^2) \neq \mathcal{O}(n\log_2 n)$.

2025 State

18.	A	The runtime of adding to a max-heap is $\mathcal{O}(\log n)$, and it is being done n times.
19.	D	Java's `PriorityQueue` utilizes a min-heap ordinarily, but because it is initialized with the `Collections.reverseOrder()` object, it is actually implementing a max-heap in this case.

Q24: A

2024 Invitational A

Q36: C

2024 Invitational B

Q32: C

2024 District

Q39: 5

2024 State

Q40: 2

2023 Invitational A

Q36: B

2023 District

Q40: 2

2023 Region

Q40: 512

2022 Invitational A

Q21: B Q29: C

2022 Invitational B

Q18: D

18.	D	Naïve Fibonacci is O(2^n) because each subproblem spawns 2 new ones
19.	A	Dynamic programming Fibonacci is o(n) as each subproblem is answered exactly once.
27.	A	DataStructure represents a Queue, so trace FIFO queue operations to produce answer A
28.	E	If the structure is empty, o will be null and o.e will cause a null pointer exception.
29.	A	Datastructure is FIFO, so it implements a queue

2022 District

Q23: C

24.	D	Each recursive call to rec spawns 2 new calls, and this occurs to a recursive depth of N. Hence runtime is 2^N
25.	B	Each value in the table is filled by referencing a constant number of other table values. the table is N*N, so runtime is N^2

Q34: C

37.	C	In the worst case, every element added to a hash table could end up in the same bucket, making contains() a linear time method. Because add() calls contains(), add's worst case performance is O(N) as well
38.	E	The structure is a hash table that uses a linkedlist to handle collisions

Q40: 12

2022 Region

		contain duplicates, only 3 unique values will be present.
21.	D	Dp and rec both find the length of the longest weakly increasing subsequence. For #21, this is 1,1,1,1,7 -> 5
22.	C	3,4,5,9 or 1,1,5,9 -> 4
23.	C	There are O(n) elements to fill in the dp table, and each one takes O(n) time to fill, resulting in O(n^2) complexity
24.	E	The complexity of brute force recursive LIS is exponential, greater than all of the given answer choices

Q29:B Q31: B Q35: D

2022 State

Q17: B

2021 Invitational A

Q40: 10011000

2021 Invitational B

		D. Will return the first occurrence.
23.	B	Class InvB implements a queue. abc adds an element to the tail and xyz removes the element at the head. mno returns but does not remove the element at the head. efg returns true if the queue is empty.
24.	A	See #23
		by zero error never occurs.
27.	E	k represents the smallest value left in the unsorted portion of the list.
28.	B	i = 0 [0, 7, 1, 8, 2, 9, 6, 5, 4, 3] i = 1 [0, 1, 7, 8, 2, 9, 6, 5, 4, 3] i = 2 [0, 1, 2, 8, 7, 9, 6, 5, 4, 3] i = 3 [0, 1, 2, 3, 7, 9, 6, 5, 4, 8] i = 4 [0, 1, 2, 3, 4, 9, 6, 5, 7, 8] i = 5 [0, 1, 2, 3, 4, 5, 6, 9, 7, 8] i = 6 [0, 1, 2, 3, 4, 5, 6, 9, 7, 8] i = 7 [0, 1, 2, 3, 4, 5, 6, 7, 9, 8] i = 8 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
29.	E	Nested loops that increment by one with each iteration typically execute in quadratic time.

Q32: D Q36: D

2021 District

Q37: b Q40:

40.	8	In a binary tree with N nodes the number of levels is at least $\log_2(N + 1)$. The base 2 logarithm of 256 is 8. $\log_2(256) = 8$

2021 Region

Q37: E Q40: O(1)

2019 Invitational A

22.	B	A binary search works by dividing the list in two on each iteration of the loop and discarding the half of the list that does not contain the target value and continuing the search in the half that does.
23.	A	The method returns the location within the list (index value) of the target value or -1 if the target is not found. In this case the index is stored in `i`.
24.	D	Reducing the size of the list by ½ on each iteration results in logarithmic time efficiency.

Q36: B

2019 Invitational B

26.	E	The method `sort` is a selection sort. Each pass of the inner loop is searching for the smallest value in the unsorted portion of the list.
27.	A	With each iteration of the outer loop the smallest value in the unsorted portion of the list is swapped with the first value in the unsorted portion of the list and then added to the sorted portion. Here is a print out of each iteration: i=0 [0, 9, 3, 7, 2, 1, 4] i=1 [0, 1, 3, 7, 2, 9, 4] i=2 [0, 1, 2, 7, 3, 9, 4] i=3 [0, 1, 2, 3, 7, 9, 4] i=4 [0, 1, 2, 3, 4, 9, 7] i=5 [0, 1, 2, 3, 4, 7, 9] i=6 [0, 1, 2, 3, 4, 7, 9]
28.	B	See #'s 26 and 27.

36.	B	Law of Union (UIL Official List of Boolean Algebra Identities)
37.	A	The `add(String s)` method does not allow duplicates.

Q40:

40.	4	A leaf within a binary search tree is any node that does not have a left or right child node. In this case, 1, 4, 60, and 82 are leaf nodes.

2019 District

Q25: B

2019 Region

Q32: B

2019 State

Q33: D

2018 Invitational A

		added to sum: 109 + 103 + 100 = 312.
32.	A	i 1 2 3 4 1 0 2 2 3 4 1 0 3 1 2 3 4 0
33.	B	An insertion sort works by choosing the next element in the array and then placing it into its proper location within the already sorted portion of the array. A common analogy is picking up playing cards from a table and placing them in order in your hand.
34.	B	Best Case O(n), Average Case O(n^2), Worst Case O(n^2)

Q38: C

2018 Invitational B

		1 10
23.	E	If ????? is replaced with `ArrayList` and the `java.util` package is imported the line with the missing code will compile, however, the lines that follow that contain the `add` method will not compile because they have a string argument.
24.	B	`ds` is a stack, which means the last element placed into `ds` will be the first one to come out. The call to `remove` pops "dust" off of the stack. `next` returns, but does not remove, the item that is on top of the stack which in this case is "hail"
25.	C	Since `ds` is a stack, elements are removed in a first in, last out fashion.
26.	A	`isEmpty` returns true when no elements are left in `ds`.
27.	D	See #24 and #25

Q37: E

2018 District

27.	B	This is an insertion sort so we are getting the next element in the unsorted portion of the array then shifting elements to the right until we find the proper place for the unsorted element or when we reach the front of the array. Then the unsorted element is placed (inserted) into the proper location.
28.	C	See #29.
29.	C	Best Case O(n), Average Case $O(n^2)$, Worst Case $O(n^2)$